ADVANCED SOFTWARE ENGINEERING

Abdulfattah Alfarra

Customer Requirements

OUTLINE

- Definition
- Advantages of a Good Requirements
- Requirements perspectives
- **Exploration Phase**
- **The Story Writing Process**
  - Write Story
  - Estimate Story
  - Split Story
- Behavior-Driven Development

Definition

- *Requirements* is the process that determines the properties a particular system should have.
- The requirements process generates the information on which the design will be based.
- For this, you have to know where a system is to be used, by whom, and what services it should provide.

Exploration Phase

- *Split a story – If Development can't estimate a whole story, or if Business realizes that part of a* story is more important than the rest, Business can split a story into two or more stories.

User Stories

What is a User Story?

A short, written description of a piece of functionality that will be valuable to a user (or owner) of the software.

It provides a simple way for developers and customers to split up what the system needs to do so the system can be delivered in pieces.

User Stories

- The emphasis on the word *simple* is essential. There are many books out there that go into great detail about requirements engineering, use case design, and similar topics. These present some useful ideas, but as ever XP looks for the simplest approach that could possibly work. So as we look at stories, remember that the whole point is to make them too simple to work in the hope that we might just get away with it.

The Story Writing Process

- The process of writing stories is iterative, requiring lots of feedback.
- Customers will propose a story to the programmers. The programmers will ask themselves if the story can be tested and estimated, and if it is of appropriate size.
- If the story cannot be estimated, the programmers will ask the customer to clarify the story. If the story is too big, the developers will ask the customer to split the story.

User Story Cards parts

User Story Cards have 3 parts

1. - A written description of the user story for planning purposes and as a reminder

2. - A section for capturing further information about the user story and details of any conversations

3. - A section to convey what tests will be carried out to confirm the user story is complete and working as expected

User Story Description

User Story Description ( )

As a ..... I want to ..... so I can .....

For example:

- As a I want so I can

User Story Description

- **Who** (user role)
- **What** (goal)
- **Why** (reason)
  - gives clarity as to why a feature is useful
  - can influence how a feature should function
  - can give you ideas for other useful features that support the user's goals

User Story Example: Front of Card

User Story Example: Back of Card

Confirmation

1. Success – valid user logged in and referred to home page.
   a. 'Remember me' ticked – store cookie / automatic login next time.
   b. 'Remember me' not ticked – force login next time.

2. Failure – display message:
   a) "Email address in wrong format"
   b) "Unrecognised user name, please try again"
   c) "Incorrect password, please try again"
   d) "Service unavailable, please try again"
   e) Account has expired – refer to account renewal sales page.

Examples of specifying value to users

- Good:
  - "A user can search for jobs"
  - "A company can post new jobs"
- Bad:
  - "The software will be written in C++"
  - "The program will connect to the database through a connection pool"

How detailed should a User Story be?

Detailed enough for the team to start work from, and further details to be established and clarified at the time of development.

Story Responsibilities (Customers)

- Writing stories that
  - Are promises to converse rather than detailed specification
  - Have value to users or to yourself
  - Are independent
  - Are testable
  - Are appropriately sized
- Have the conversations with the developers

Story Responsibilities (Developers)

- Help the customers write stories that
  - Are promises to converse rather than detailed specs
  - Have value to the users or the customer
  - Are independent
  - Are testable
  - Are appropriately sized
- Describing the need for technology/infrastructure in terms of value to users or customers
- Have the conversations with the customers

Users

- Can be more than one type of user for system
- Different user types may have different roles and different stories
- Disfavored users (e.g., hackers) may be included as well as favored users
  - Obviously, the value to users piece gets flipped for disfavored users

Example

- Travel booking project
- **Find lowest fare.**
  - *Present to the customer the ten lowest fares for a particular route.*
- **Show available flights.**
  - *Show possible flights (with connections) between any two planets.*
- **Sort available flights by convenience**.
  - *When you're showing the flights, sort them by convenience: time of journey, number of changes, closeness to desired departure and arrival time.*
- **Purchase ticket**.
  - *Purchase ticket charging to a credit card. Check credit card validity when doing this.*
- **Do customer profile.**
  - *Keep customer details for quick reference, for example, credit card info, home address, dietary and gravitational needs*

Another Example

- **Review journeys.**
  - *Show all* **journeys** *the customer has in the system.*
- **Cancel journey.**
  - *If a customer cancels an journey, cancel all flights, hotels, etc.*
- **Print immigration paperwork.**
  - *Print paperwork required to leave and arrive at a planet).*
- **Show hotels**.
  - *Show hotels near a place.*
- **Show hotel availability.**
  - *Show hotels that are available for the period indicated by the journey.*
- **Offer sophisticated hotel search.**
  - *Allow customer to search for hotels using more than dates and location. This would include facilities, level of service, costs, and recommendations.*
- **Book a hotel.**
  - *Book a hotel. Charge to credit card and check credit card validity.*

Non-functional Stories

- System must be easy to use
- System must be able to support 30,000 users
- System runs on Windows, Linux, and BeOS
- No one can steal anyone else's data

Non-functional Stories

- Some need up-front attention
  - Security
- Some just need to be remembered:
  - Performance
  - Easy to use
- Some need continuing attention
  - Portability
  - Easy to use
- Remember Refactoring

User Stories are Not

- Requirements documents
- Use Cases
- Scenarios

Not good user stories

- Stories are too small
- Dependent stories
- Too Many details
- Including UI detail too soon
- Thinking too far ahead
- Splitting too many stories
- Customer has trouble prioritizing
- Customer won't write and prioritize the stories

Story Estimation

- Estimation is one of the harder parts of a software project.
- *Development estimates how long the story will take to implement. If* Development can't estimate the story, it can ask Business to clarify or split the story.
- Examples of Methods for estimating stories are: By analogy or by Planning Poker

Estimate by analogy

- Comparing a user story to others
  - “This story is like that story, so its estimate is what that story’s estimate was.”
- Compare the story being estimated to multiple other stories.

Planning Poker

- *The best way found for agile teams to estimate is by playing planning poker*
- This method tries to make the meetings more short and productive, by making them more fun and dynamic.

Preparing the meeting

- The “requirements experts” must know perfectly each of the user stories.
- Each user story should have a granularity of no more than 10 days of job.
- A deck of cards is prepared for each member of the team.
- The deck is composed of a few cards, each of them representing a estimation.

Preparing the meeting

- Examples of estimation values for the cards:
  - 0, 1, 2, 3, 5, 8, 13, 20, 40, 100.
  - 1, 2, 3, 5, 8, 13, BIG.
  - ½, 1, 2, 3, 4, 5, 6, 7, ∞

The meeting

1. A deck is given to each of the members.
2. The moderator exposes a user story in no more than 2 minutes.
3. Time for questions about the user story.
4. Each of the members choose a card privately.
5. Once everybody has chosen, all the cards are turned over at the same time.
6. In this first round, it's probably that the estimations will differ significantly.

The meeting

7. In case the estimations differ, the high and low estimators expose their reasons.
8. A few minutes for the team to discuss about the story and the estimation.
9. Again, each member thinks privately a estimation, and they show the cards simultaneously.
10. If the estimations still differ, the same process can be repeated.

The meeting

11. When the estimations converge, the process finishes and the next user story is estimated.
12. In case the estimations don't converge by the 3rd round, there are some options:
    - Left the user story apart and try again later.
    - Ask the user to decompose the story in smaller parts.
    - Take the highest, lowest or average estimation.

Example (I)

- User story: PCGEEK wants to be able to create sell orders.
- Team of 7 members.
- First round:

Example (II)

- 3rd and 6th members expose their reasons for their estimations.
- 2nd round:

Example (III)

- All members have converged except for the 3rd
- A new round of expositions and voting can be made.
- It's also possible to take 3 or 5 as the estimation.

Splitting Stories

- Compound
  - Conversations may reveal multiple stories
  - Split along Create/Update/Delete
  - Split along data boundaries
- Complex
  - Split into investigative and develop the new feature stories
- Make sure each split-off portion is a good story

Example

- As a salon owner I want to be able to sell items on a ticket so I can generate revenue.

(Split to)

- As a salon owner I want to be able to sell services on a sales ticket so I can generate revenue.
- As a salon owner I want to be able to sell products on a sales ticket so I can generate revenue.
- As a salon owner I want to be able to sell gift certificates on a sales ticket so I can generate revenue.

Another Example

- As a salon owner I want to be able to sell services on a sales ticket so I can generate revenue.

(Split to)

  - As a salon owner I want to be able to sell individual services on a sales ticket so I can generate revenue.
  - As a salon owner I want to be able to sell packages so I can increase revenue.

Behavior-Driven Development

- Behavior-Driven Development is about implementing an application by describing its behavior from the perspectives of its stakeholders

- Behavior-driven development is a set of best practices that advise you on how to develop software by centering your users.

Example

- **Specification:** Stack

When a new stack is created

Then it is empty

When an element is added to the stack

Then that element is at the top of the stack

When a stack has N elements **And** element E is on top of the stack

Then a pop operation returns E **And** the new size of the stack is N-1

Behavior-Driven Development

- Behavior Driven Development combines requirements and testing into one integrated approach resulting in improved understanding by both the business and the technical team

Behavior-Driven Development

- **Benefit**
- BDD is helpful in communicating requirements effectively. Requirements are specified in a way that they are explicit, executable and testable. A common language is used to allow the business, development and testing staff to understand requirements, thus facilitating communication with less ambiguity.

Behavior-Driven Development

- Each user story provides a description of a feature that provides value to a user.
- Once a story has been identified we then focus on the scenarios that describe how the user expects the system to behave using a sequence of steps

Behavior-Driven Development

- The goal is to build requirement Scenarios using a series of examples. These examples support the creation of a common language that everyone on the team can use to gain a common agreement on the objectives of the system being built.

Behavior-Driven Development

- Scenarios document the conversation took place

  Given <some initial context>

  When <this happens>

  Then <this should be the result>

Behavior-Driven Development

- Scenarios provide a narrative described the sequencing of events necessary to complete specific Acceptance Criteria. An example of scenario following the BDD format is :
- It starts by specifying the initial condition that is assumed to be true at the beginning of the scenario. This may consist of a single clause, or several.
- It then states which event triggers the start of the scenario.
- Finally, it states the expected outcome, in one or more clauses.

Example

- Feature: Daily car maintenance

As a car owner

I want to be able to fuel my car

So I can get where I want

Scenario: Fuelling

a car with 10 liters of fuel in the tank

you fill it with 50 liters of fuel

the tank contains 60 liters

Example

- Feature Transfer money from savings to current Account
- Scenario: Savings account has enough money

  my savings account balance is 100

  my current account balance is 50

  I transfer 20 from savings to current

  my savings account balance should be 80

  my current account balance should be 70

Example

```
Feature: Benefit Payment
  In order for Participants to receive Benefit Payments,
  Participant must have paid $75 fee by September 30 of the Program Year.

Scenario: Participant who meet all benefit criteria should be paid
      Given a Participant has not paid any fee
      When $75 fee is paid on September 1
      Then on October 1 the Benefit Paid status on should true

Scenario: Participant who do not pay enough fee does not get benefit
      Given a Participant has not paid any fee
      When $70 fee is paid on September 1
      Then on November 1 the Benefit Paid status on should false

Scenario: Late paying Participant does not get benefit
      Given a Participant has not paid any fee
      When $75 fee is paid on October 1
      Then on November 1 the Benefit Paid status on should false
```